



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251172
Nama Lengkap	Ang, Joshua Alexander Hartono
Minggu ke / Materi	12 / Tipe Data Dictionary

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pengantar

Dictionary pada Python mirip seperti list karena sama-sama digunakan untuk menyimpan banyak data dalam satu variabel. Perbedaannya adalah list menggunakan indeks angka atau integer untuk mengambil sebuah data, sedangkan pada dictionary menggunakan key untuk mengambil data. Di dalam dictionary sendiri, data disimpan kedalam bentuk berpasang pasangan yaitu key dan value. Key berfungsi sebagai nama untuk mengambil suatu nilai (value) tertentu. Oleh karena itu, key harus unik dan tidak boleh sama dalam satu dictionary.

Dictionary juga bisa dianggap sebagai pemetaan antara key dan value, yang artinya setiap key pasti akan terhubung dengan satu nilai tertentu. Hal pertama yang harus dilakukan adalah membuat sebuah dictionary. Untuk membuat sebuah dictionary ada dua cara yang dapat dilakukan. Pertama menggunakan fungsi `dict()` untuk membuat dictionary kosong atau dengan tanda kurung kurawal `{}`. Setelah dictionary berhasil dibuat maka data bisa ditambahkan dengan tanda kurung siku `[]` menggunakan format `nama_dictionary[key] = value`.

```
>>> eng2sp = dict()
>>> print(eng2sp)
{}
```

Gambar 12.1: Cara membuat dictionary kosong. *Sumber:* <https://shorturl.at/64Fki>

```
>>> eng2sp['one'] = 'uno'
```

Gambar 12.2: Contoh menambahkan item pada dictionary. *Sumber:* <https://shorturl.at/64Fki>

Saat dictionary dicetak, seluruh pasangan key dan value akan tampil dalam format yang hampir sama seperti saat penulisannya. Dictionary juga dapat langsung dibuat beserta isinya, misalnya berisi beberapa pasangan kata sekaligus. Namun urutan data pada dictionary tidak selalu tetap atau berurutan seperti list. Hal ini dikarenakan dictionary tidak bekerja berdasarkan indeks angka tetapi berdasarkan dari key.

```
>>> eng2sp = {'one': 'uno', 'two': 'dos', 'three': 'tres'}
>>> print(eng2sp)
{'one': 'uno', 'three': 'tres', 'two': 'dos'}
```

Gambar 12.3: Contoh menambahkan banyak item pada dictionary. *Sumber:*

<https://shorturl.at/64Fki>

Untuk mengambil nilai dari dictionary, kita cukup menuliskan nama key di dalam kurung siku. Seperti pada gambar di atas, jika kita menggunakan key “two” maka akan menghasilkan nilai “dos”. Tetapi, jika key yang dicari tidak ada dalam dictionary maka Python akan menghasilkan error berupa `KeyError`. Selain itu, fungsi `len()` juga dapat digunakan pada dictionary untuk menghitung jumlah dari pasangan key dan value yang ada. Operator `in` pada dictionary digunakan untuk mengecek apakah suatu key ada di dalam dictionary tersebut atau tidak. Untuk mengecek hal yang sama pada value dapat menggunakan method `values()` terlebih dahulu lalu mengubahnya ke list.

Dictionary sebagai set penghitung (counters)

Dictionary dapat digunakan sebagai counter untuk menghitung berapa kali suatu kata muncul. Pada materi ini contohnya adalah menghitung jumlah huruf dalam sebuah string. Ada beberapa cara yang dapat dilakukan, misalnya adalah membuat banyak variabel untuk setiap huruf alfabet atau menggunakan list dengan indeks angka. Namun cara tersebut kurang fleksibel karena program harus menyimpan tempat untuk kemungkinan semua huruf.

Pada penggunaan dictionary, setiap huruf dijadikan sebagai key dan jumlah kemunculan dijadikan value. Program akan membaca string satu persatu menggunakan perulangan `for`. Jika huruf belum ada di dalam dictionary, maka program akan membuat key baru dengan nilai awal 1. Tetapi jika huruf tersebut sudah ada, maka nilainya akan ditambah satu lagi. Proses ini akan terus berjalan sampai seluruh karakter huruf diperiksa.

```
1 word = 'brontosaurus'
2 d = dict()
3 for c in word:
4     if c not in d:
5         d[c] = 1
6     else:
7         d[c] = d[c] + 1
8 print(d)
```

Gambar 12.4: Contoh penggunaan dictionary sebagai counter untuk mencari jumlah huruf yang ada pada sebuah kata. *Sumber:* <https://shorturl.at/64Fki>

Selain itu, terdapat juga method `get()` pada dictionary yang digunakan untuk mengambil nilai dari suatu key. Jika key yang dicari ditemukan, maka nilai aslinya akan dikembalikan. Namun, jika key tidak ada, maka method `get()` akan memberikan nilai `None` atau kita dapat memberikan nilai default yang sudah ditentukan. Penggunaan `get()` membuat program menjadi lebih singkat karena tidak perlu lagi menggunakan percabangan `if` untuk mengecek keberadaan key seperti gambar 12.3.

```
1 word = 'brontosaurus'
2 d = dict()
3 for c in word:
4     d[c] = d.get(c,0) + 1
5 print(d)
```

Gambar 12.5: Contoh penggunaan `get()` pada dictionary sebagai counter untuk mencari jumlah huruf yang ada pada sebuah kata. *Sumber:* <https://shorturl.at/64Fki>

Dictionary dan file

Salah satu penggunaan dictionary pada Python adalah menghitung jumlah kata pada sebuah file teks. Program akan membaca isi file satu per satu lalu menghitung berapa kali setiap kata muncul. Dengan bantuan dictionary, setiap kata dapat disimpan sebagai key dan value.

Program menggunakan dua perulangan. Pada perulangan pertama digunakan untuk membaca setiap baris pada file, sedangkan perulangan kedua digunakan untuk membaca setiap kata pada baris tersebut. Setelahnya, method `split()` digunakan untuk memisahkan kalimat menjadi beberapa kata dalam bentuk list. Kemudian setiap kata diperiksa apakah sudah ada di dictionary. Jika belum, maka kata tersebut akan ditambahkan sebagai key dengan nilai 1, sedangkan jika kata sudah ada, maka nilainya akan ditambah dengan satu.

Pada bagian program juga digunakan penulisan singkat `counts[word] += 1`. Penulisan ini sama dengan `counts[word] = counts[word] + 1`. Selain itu, penggunaan metode lain seperti `-=`, `*=`, `/=` dapat digunakan mempermudah perubahan nilai variabel.

```

1 fname = input('Enter the file name: ')
2 try:
3     fhand = open(fname)
4 except:
5     print('File cannot be opened:', fname)
6     exit()
7
8 counts = dict()
9 for line in fhand:
10     words = line.split()
11     for word in words:
12         if word not in counts:
13             counts[word] = 1
14         else:
15             counts[word] += 1
16
17 print(counts)

```

Gambar 12.6: Penggunaan dictionary dalam mencari jumlah kata pada sebuah file. *Sumber:*

<https://shorturl.at/64Fki>

Looping dan Dictionary

Pada dictionary, perulangan for digunakan untuk menelusuri setiap key yang ada di dalam dictionary. Saat loop berjalan, variabel pada for akan berisi key satu per satu. Untuk mengambil nilai dari key tersebut, program perlu menggunakan indeks dictionary seperti counts[key]. Dengan cara ini, program dapat menampilkan pasangan antara key dan value secara bersamaan.

```

1 counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
2 for key in counts:
3     print(key, counts[key])

```

Outputnya akan tampak sebagai berikut:

```

jan 100
chuck 1
annie 42

```

Gambar 12.7: Contoh penggunaan for pada dictionary. *Sumber:* <https://shorturl.at/64Fki>

Adapun cara untuk mengurutkan isi dictionary berdasarkan key secara alfabet. Langkah pertama adalah mengambil semua key menggunakan method keys(), lalu mengubahnya menjadi list dengan fungsi list(). Setelah itu diurutkan menggunakan sort(). Kemudian program akan

melakukan looping pada list yang sudah terurut tersebut dan mencetak pasangan key serta value sesuai urutan alfabet.

```
1 counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
2 lst = list(counts.keys())
3 print(lst)
4 lst.sort()
5 for key in lst:
6     print(key, counts[key])
```

Outputnya sebagai berikut:

```
['jan', 'chuck', 'annie']
annie 42
chuck 1
jan 100
```

Gambar 12.8: Program untuk mengurutkan isi dictionary. *Sumber:* <https://shorturl.at/64Fki>

Advanced Text Parsing

Sebelumnya, file teks yang digunakan sudah dibersihkan dari tanda baca sehingga proses perhitungan kata menjadi lebih mudah. Namun pada kasus nyata, file biasanya masih memiliki tanda baca. Contohnya kata “soft!” dan “soft!” akan dihitung sebagai dua kata yang berbeda. Selain tanda baca, penggunaan huruf kapital juga mempengaruhi hasil perhitungan kata. Python membedakan huruf besar dan huruf kecil sehingga “who” dan “Who” dianggap berbeda. Akibatnya hasil perhitungan kata bisa menjadi tidak akurat karena kata yang sama dipisahkan menjadi dua kata. Oleh karena itu, method `lower()` digunakan untuk mengubah semua huruf menjadi huruf kecil.

Selain itu, method `translate()` digunakan untuk menghapus tanda baca pada teks. Python menyediakan daftar tanda baca melalui `string.punctuation`. kemudian method `translate()` dipakai bersama `maketrans()` untuk menghapus semua karakter tanda baca pada setiap baris teks. Sehingga tanda baca dihapus dan huruf diubah menjadi kecil, barulah program menggunakan `split()` untuk memisahkan kata-kata berdasarkan spasi. Selanjutnya program akan menghitung jumlah kemunculan kata menggunakan dictionary

```

1 import string
2
3 fname = input('Enter the file name: ')
4 try:
5     fhand = open(fname)
6 except:
7     print('File cannot be opened:', fname)
8     exit()
9
10 counts = dict()
11 for line in fhand:
12     line = line.rstrip()
13     line = line.translate(line.maketrans('', '', string.punctuation))
14     line = line.lower()
15     words = line.split()
16     for word in words:
17         if word not in counts:
18             counts[word] = 1
19         else:
20             counts[word] += 1
21
22 print(counts)

```

Output dari program diatas adalah :

```

Enter the file name: romeo-full.txt
{'swearst': 1, 'all': 6, 'afeard': 1, 'leave': 2, 'these': 2,
'kinsmen': 2, 'what': 11, 'thinkst': 1, 'love': 24, 'cloak': 1,
a': 24, 'orchard': 2, 'light': 5, 'lovers': 2, 'romeo': 40,
'maiden': 1, 'whiteupturned': 1, 'juliet': 32, 'gentleman': 1,
'it': 22, 'leans': 1, 'canst': 1, 'having': 1, ...}

```

Gambar 12.9: Program untuk menghitung kata yang ada dalam sebuah file dengan adanya tanda baca dan huruf kapital. *Sumber:* <https://shorturl.at/64Fki>

BAGIAN 2: LATIHAN MANDIRI (60%)

Link github : <https://github.com/angjoshua-sudo/Prak-Alpro-2026.git>

SOAL 1

Source code:

```
1 dictionary = eval(input("Masukkan angka: "))
2 print("Key Value Item")
3 for key, value in dictionary.items():
4     print(f"{key} {value} {key}")
```

Penjelasan:

Pada bagian awal program, pada baris pertama program akan meminta input kepada user berupa sebuah dictionary dan akan disimpan ke variabel **dictionary**. Input menggunakan fungsi **eval()** sehingga input yang diberikan user akan dibaca sebagai tipe data Python. Selanjutnya pada baris kedua, program akan menampilkan teks **"Key Value Item"** sebagai judul dari output. Kemudian pada baris ketiga, program akan menggunakan perulangan **for** untuk mengambil isi dari dictionary menggunakan method **.items()**. Method ini digunakan agar dapat mengambil pasangan dari **key** dan **value** yang ada di setiap iterasi. Pada baris terakhir, program akan mencetak hasil menggunakan **fstring** agar sesuai dengan format output terutama dalam hal spasi. Di dalam output tersebut, program akan menampilkan **key, value, key**.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github\mingui2> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/mingui2/soal1.py
Masukkan angka: {1: 10, 2: 20, 3: 30, 4: 40, 5: 50, 6: 60}
Key Value Item
1 10 1
2 20 2
3 30 3
4 40 4
5 50 5
6 60 6
```


SOAL 2

Source code:

```
1  lista = eval(input("Masukkan key: "))
2  listb = eval(input("Masukkan value: "))
3
4  dictionary = {}
5
6  for i in range(len(lista)):
7      dictionary[lista[i]] = listb[i]
8
9  print(dictionary)
```

Penjelasan:

Pada bagian awal program, pada baris pertama dan kedua, program akan meminta input dari user berupa dua buah list yaitu **lista** dan **listb**. List pertama digunakan untuk menampung key dan list kedua digunakan untuk menampung value. Kedua input menggunakan **eval()** agar input dari user dapat masuk ke dalam bentuk list Python. Selanjutnya pada baris keempat terdapat dictionary kosong bernama **dictionary**. Dictionary ini berfungsi untuk menyimpan nilai key dan value yang nanti akan ditambahkan. Kemudian pada baris keenam, program menggunakan perulangan **for** dengan **range(len(lista))** untuk melakukan iterasi berdasarkan jumlah elemen pada list key. Pada baris ketujuh, program akan mulai mengisi dictionary dengan format **dictionary[lista[i]] = listb[i]**. Sehingga lista akan menjadi key dan listb akan menjadi value. Terakhir baris kesembilan akan mencetak isi dictionary.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu_12/soal2.py"
Masukkan key: ['red', 'green', 'blue']
Masukkan value: ['#FF0000', '#008000', '#0000FF']
{'red': '#FF0000', 'green': '#008000', 'blue': '#0000FF'}
```

SOAL 3

Source code:

```
1 file = input("Masukkan nama file: ")
2 try:
3     handle = open(file)
4 except:
5     print("File tidak ditemukan")
6     exit()
7
8 mail = {}
9
10 for line in handle:
11     if line.startswith("From "):
12         kata = line.split()
13         email = kata[1]
14         mail[email] = mail.get(email, 0) + 1
15
16 print(mail)
```

Penjelasan:

Pada bagian awal, program akan meminta user memasukkan nama file yang akan disimpan di variabel **file**. Kemudian program akan mencoba membuka file menggunakan **try** dan **except**. Jika berhasil dibuka, program akan melanjutkan proses pembacaan file. Jika tidak maka program mencetak tulisan "**File tidak ditemukan**" lalu menghentikan program menggunakan **exit()**. Setelah itu, program akan membuat dictionary kosong bernama **mail**.

Selanjutnya program membaca file perbaris menggunakan perulangan **for line in handle**. Program hanya akan memproses baris yang diawali dengan "**From** " menggunakan fungsi **startswith()** karena baris tersebut berisi alamat email pengirim. Setiap baris yang sesuai akan dipisahkan menggunakan **split()**, kemudian email diambil dari indeks ke-1 dan disimpan ke variabel **email**. Setelahnya program akan menghitung kemunculan email tersebut menggunakan dictionary dengan method **get()**. Terakhir, program akan mencetak isi dictionary **mail**.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github\mingui2> py soal3.py
Masukkan nama file: mbox-short.txt
{'stephen.marquard@uct.ac.za': 2, 'louis@media.berkeley.edu': 3, 'zqian@umich.edu': 4, 'rjlowe@iupui.edu': 2, 'cwen@iupui.edu': 5, 'gsilver@umich.edu': 3, 'wagnermr@iupui.edu': 1, 'antranig@caret.cam.ac.uk': 1, 'gopal.ramasammycook@gmail.com': 1, 'david.horwitz@uct.ac.za': 4, 'ray@media.berkeley.edu': 1}
```

SOAL 4

Source code:

```
1 file = input("Masukkan nama file: ")
2 try:
3     handle = open(file)
4 except:
5     print("File tidak ditemukan")
6     exit()
7
8 mail = {}
9
10 for line in handle:
11     if line.startswith("From "):
12         kata = line.split()
13         email = kata[1]
14         email = email.split("@")[1]
15         mail[email] = mail.get(email, 0) + 1
16
17 print(mail)
```

Penjelasan:

Pada bagian awal, program akan meminta user memasukkan nama file yang akan disimpan di variabel **file**. Kemudian program akan mencoba membuka file menggunakan **try** dan **except**. Jika berhasil dibuka, program akan melanjutkan proses pembacaan file. Jika tidak maka program mencetak tulisan "**File tidak ditemukan**" lalu menghentikan program menggunakan **exit()**. Setelah itu, program akan membuat dictionary kosong bernama **mail**.

Selanjutnya program membaca file perbaris menggunakan perulangan **for line in handle**. Program hanya akan memproses baris yang diawali dengan "**From** " menggunakan fungsi **startswith()** karena baris tersebut berisi alamat email pengirim. Setiap baris yang sesuai akan dipisahkan menggunakan **split()**, kemudian email diambil dari indeks ke-1 dan disimpan ke variabel **email**. Setelahnya program kembali menggunakan **split("@")** untuk memisahkan username dan domain email lalu mengambil domain email menggunakan indeks **[1]**. Program kemudian akan menghitung kemunculan email tersebut menggunakan dictionary dengan method **get()**. Terakhir, program akan mencetak isi dictionary **mail**.

Output:

```
● PS C:\Users\Acer\OneDrive\Dokumen\belajar-github\minggu12> py soal4.py
Masukkan nama file: mbox-short.txt
{'uct.ac.za': 6, 'media.berkeley.edu': 4, 'umich.edu': 7, 'iupui.edu': 8, 'caret.cam.ac.uk': 1, 'gmail.com': 1}
```

++